

SQL per prove TdP - metodi, costrutti e template estesi

Guida pratica per query di nodi, archi, pesi, statistiche e DAO

Come usare questo file

- Usalo come foglio da esame: individua nodi, archi, peso e filtri temporali, poi scegli il template piu simile.
- Se devi creare coppie di nodi, quasi sempre serve un self-join: la stessa tabella o CTE usata due volte.
- Per grafo non orientato usa $id1 < id2$ oppure LEAST/GREATEST; per grafo orientato non ordinare mai i due nodi.
- Per il peso scegli tra COUNT, COUNT(DISTINCT), SUM, MIN/MAX o una CTE con statistiche gia calcolate.
- Prima fai uscire id1-id2 senza peso; solo dopo aggiungi GROUP BY e aggregato.

Regola fondamentale: i nodi vanno presi con una query separata dagli archi, soprattutto se la traccia dice che alcuni nodi possono rimanere isolati.

1. Struttura base di una query

Parte	A cosa serve	Ordine logico
SELECT	Sceglie le colonne da restituire	Dopo i filtri, ma si scrive per prima
FROM	Indica tabelle e alias	Punto di partenza
JOIN / WHERE	Collega tabelle e filtra righe	Prima del raggruppamento
GROUP BY	Crea gruppi per calcolare pesi/statistiche	Dopo WHERE
HAVING	Filtra i gruppi gia aggregati	Dopo GROUP BY
ORDER BY	Ordina il risultato	Quasi alla fine
LIMIT	Tiene solo le prime N righe	Ultimo passaggio

```
SELECT colonne, aggregati
FROM tabella1 t1, tabella2 t2
WHERE t1.id = t2.id_fk
  AND filtro
GROUP BY colonne_non_aggregate
HAVING filtro_sugli_aggregati
ORDER BY aggregato DESC
LIMIT 5;
```

2. Costrutti principali: cosa fanno e quando usarli

Costrutto	Cosa restituisce / fa	Quando usarlo nelle prove
AS	Crea alias di colonna o tabella	Rendere chiari id1, id2, peso.
DISTINCT	Elimina duplicati identici	Nodi unici, elementi condivisi contati una volta.
WHERE	Filtra righe prima del GROUP BY	Range, date, genere, categoria, rating.
AND / OR / NOT	Combina condizioni logiche	Filtri multipli; usa parentesi con OR.
IN (...)	Verifica appartenenza a lista/sottoquery	Nodi validi presi da query precedente.
NOT IN (...)	Esclude valori	Rimuovere nodi non ammessi; attenzione ai NULL.
IS NULL / IS NOT NULL	Controlla valori mancanti	Date di nascita valide, incassi presenti, posizioni non nulle.
LIKE	Cerca testo con pattern	Filtri su nome, stato, descrizione.
BETWEEN	Range inclusivo	Date/rating/prezzi con estremi inclusi.
> / < / >= / <=	Range esclusivi o confronti tra nodi	Anni estremi esclusi, dal maggiore al minore.
INNER JOIN	Tiene solo righe che matchano	Equivalente ai join nel WHERE, piu leggibile.
LEFT JOIN	Tiene anche righe senza match	Nodi isolati o conteggi zero.
SELF JOIN	Confronta una tabella con se stessa	Coppie attore-attore, album-album, paese-paese.
WITH CTE	Tabella temporanea leggibile	Spezzare query lunghe: nodi validi, statistiche, acquisti.
UNION	Unisce risultati eliminando duplicati	Inserire archi doppi in parita o casi separati.
UNION ALL	Unisce risultati mantenendo duplicati	Quando vuoi preservare tutte le righe.
CASE WHEN	Crea valori condizionali	Calcolare direzione, etichette, categorie.
COALESCE	Sostituisce NULL con un valore	Conteggi zero, incassi mancanti gestiti.
COUNT(*)	Conta righe	Numero eventi/co-occorrenze.
COUNT(DISTINCT x)	Conta valori distinti	Numero film/artisti/clienti condivisi.

SUM(x)	Somma valori	Pesi come incassi, quantita, vendite.
AVG/MIN/MAX	Media, minimo, massimo	Statistiche su rating/date/prezzi.
ROUND(x, n)	Arrotonda	Stampare medie piu leggibili.
ABS(x)	Valore assoluto	Peso come distanza/differenza.
DATE(x)	Estrae la data da un timestamp	Confrontare con YYYY-MM-DD.
YEAR/MONTH	Estrae anno/mese	Dropdown o filtri temporali.
DATEDIFF	Differenza in giorni	Archi tra eventi entro K giorni.
ORDER BY	Ordina righe	Top archi, anni decrescenti, dropdown.
LIMIT	Limita numero righe	Primi 5 archi/statistiche.
EXISTS	Verifica esistenza di almeno una riga	Condizioni "almeno uno" senza contare tutto.

3. SELECT, alias e DISTINCT

```
-- Alias di tabella e colonne chiare per il DAO
SELECT p.product_id AS id,
       p.product_name AS nome
FROM products p;

-- Nodi senza duplicati
SELECT DISTINCT a.AlbumId, a.Title
FROM Album a, Track t
WHERE a.AlbumId = t.AlbumId
      AND t.GenreId = %s;

-- DISTINCT su piu colonne: elimina duplicati della coppia completa
SELECT DISTINCT customer_id, artist_id
FROM acquisti;
```

DISTINCT non vuol dire "distinct solo sulla prima colonna": considera tutta la riga selezionata.

4. WHERE: filtri principali

```
-- Filtro semplice
WHERE t.GenreId = %s

-- Piu filtri insieme
WHERE r.avg_rating BETWEEN %s AND %s
      AND n.date_of_birth IS NOT NULL
      AND m.worlwide_gross_income IS NOT NULL

-- OR con parentesi
WHERE (c.Country = "USA" OR c.Country = "Canada")
      AND i.Total > 10

-- IN con lista
WHERE c.Country IN ("USA", "Canada", "Brazil")

-- LIKE: % significa qualunque sequenza di caratteri
WHERE a.Name LIKE "%Queen%"
```

Con OR metti quasi sempre le parentesi, altrimenti AND e OR possono produrre risultati diversi da quelli attesi.

5. JOIN: stile WHERE e stile esplicito

Nelle prove spesso si usa lo stile con FROM t1, t2 e condizioni nel WHERE. Lo stile JOIN esplicito e equivalente ma piu leggibile.

```
-- Stile classico usato spesso nelle esercitazioni
SELECT il.InvoiceLineId, t.TrackId
FROM InvoiceLine il, Track t
WHERE il.TrackId = t.TrackId;

-- Stessa cosa con INNER JOIN
SELECT il.InvoiceLineId, t.TrackId
FROM InvoiceLine il
INNER JOIN Track t ON il.TrackId = t.TrackId;

-- LEFT JOIN: tiene anche i nodi senza righe collegate
SELECT p.product_id,
       COUNT(oi.order_id) AS vendite
FROM products p
LEFT JOIN order_items oi ON p.product_id = oi.product_id
GROUP BY p.product_id;
```

Se devi mantenere nodi isolati o conteggi zero, valuta LEFT JOIN. Se usi WHERE sulla tabella a destra del LEFT JOIN, rischi di trasformarlo di nuovo in INNER JOIN.

6. NULL: valori mancanti

```
-- Corretto
WHERE n.date_of_birth IS NOT NULL

-- Sbagliato
WHERE n.date_of_birth != NULL

-- Sostituisco NULL con 0
SELECT p.id, COALESCE(v.num_vendite, 0) AS num_vendite
FROM prodotti p
LEFT JOIN vendite v ON p.id = v.id;
```

Con NULL non usare = o !=. Usa IS NULL oppure IS NOT NULL.

7. Aggregati e GROUP BY

Aggregato	Risultato	Uso tipico
COUNT(*)	Conta tutte le righe del gruppo	Numero co-occorrenze.
COUNT(colonna)	Conta valori non NULL	Conteggio ignorando NULL.
COUNT(DISTINCT colonna)	Conta valori diversi	Film/artisti/clienti condivisi.
SUM(colonna)	Somma numerica	Incassi, quantita, vendite.
AVG(colonna)	Media	Rating medio, prezzo medio.
MIN/MAX(colonna)	Minimo/massimo	Prima/ultima data, peso massimo.
ROUND(AVG(x),2)	Media arrotondata	Output ordinato nel testo.
GROUP_CONCAT(x)	Concatena valori in stringa	Debug: vedere elementi condivisi.

```
-- Regola: le colonne non aggregate nella SELECT devono stare nel GROUP BY
SELECT rm1.name_id AS id1,
       rm2.name_id AS id2,
       COUNT(DISTINCT rm1.movie_id) AS peso
FROM role_mapping rm1, role_mapping rm2
WHERE rm1.movie_id = rm2.movie_id
  AND rm1.name_id < rm2.name_id
GROUP BY rm1.name_id, rm2.name_id;

-- HAVING filtra dopo aver calcolato il COUNT
HAVING COUNT(DISTINCT rm1.movie_id) >= 2;
```

WHERE filtra righe prima del conteggio; HAVING filtra gruppi dopo il conteggio.

8. Date e tempo

```
-- Range inclusivo: estremi inclusi
WHERE DATE(o.order_date) BETWEEN "2016-01-01" AND "2018-12-28"

-- Range esclusivo: estremi esclusi
WHERE r.year > %s AND r.year < %s

-- Entro K giorni, evitando duplicati invertiti
WHERE o1.order_date < o2.order_date
  AND DATEDIFF(o2.order_date, o1.order_date) <= %s

-- Estrazione anno/mese
WHERE YEAR(o.order_date) = %s
WHERE MONTH(o.order_date) = %s

-- Ordinamento cronologico
ORDER BY o.order_date ASC;
```

Le date vanno tra apici. 2016-05-16 senza apici viene interpretato come sottrazione: 2016 - 5 - 16.

9. Archi non orientati: coppie senza duplicati

```
-- Metodo preferito: confronto diretto tra id
SELECT a.id AS id1, b.id AS id2
FROM nodi a, nodi b
WHERE a.id < b.id;
```

```
-- Metodo utile se la coppia nasce gia disordinata
SELECT LEAST(x.idA, x.idB) AS id1,
       GREATEST(x.idA, x.idB) AS id2,
       COUNT(*) AS peso
FROM ... x
WHERE x.idA <> x.idB
GROUP BY LEAST(x.idA, x.idB), GREATEST(x.idA, x.idB);
```

Per grafo non orientato evita a.id != b.id, perche produce sia A-B sia B-A. Usa a.id < b.id.

10. Pattern piu comune: due nodi hanno X in comune

```
-- Template generale
WITH elementi AS (
    SELECT DISTINCT nodo_id, elemento_condiviso
    FROM ...
    WHERE ...
)
SELECT e1.nodo_id AS id1,
       e2.nodo_id AS id2,
       COUNT(DISTINCT e1.elemento_condiviso) AS peso
FROM elementi e1, elementi e2
WHERE e1.elemento_condiviso = e2.elemento_condiviso
      AND e1.nodo_id < e2.nodo_id
GROUP BY e1.nodo_id, e2.nodo_id;

-- Esempio attori: X = film
SELECT rml.name_id AS id1,
       rm2.name_id AS id2,
       COUNT(DISTINCT rml.movie_id) AS peso
FROM role_mapping rml, role_mapping rm2
WHERE rml.movie_id = rm2.movie_id
      AND rml.name_id < rm2.name_id
GROUP BY rml.name_id, rm2.name_id;
```

11. Archi orientati: verso deciso da una statistica

```
-- Dal nodo con valore maggiore al nodo con valore minore
WITH stat AS (
    SELECT id_nodo AS id,
           COUNT(DISTINCT evento_id) AS valore
    FROM ...
    GROUP BY id_nodo
)
SELECT s1.id AS source,
       s2.id AS target,
       s1.valore + s2.valore AS peso
FROM stat s1, stat s2
WHERE s1.id <> s2.id
      AND s1.valore > s2.valore;

-- Se in parita si inseriscono entrambi gli archi
WITH stat AS (
    SELECT id_nodo AS id,
           COUNT(DISTINCT evento_id) AS valore
    FROM ...
    GROUP BY id_nodo
)
SELECT s1.id AS source,
       s2.id AS target,
       s1.valore + s2.valore AS peso
FROM stat s1, stat s2
WHERE s1.id <> s2.id
      AND s1.valore >= s2.valore;
```

Nel grafo orientato non usare LEAST/GREATEST: source e target devono rispettare la regola della traccia.

12. Parita negli archi orientati: alternativa con UNION

```
WITH stat AS (
    SELECT id, COUNT(*) AS valore
    FROM ...
    GROUP BY id
)
-- Caso valore diverso: solo maggiore -> minore
```

```

SELECT s1.id AS source, s2.id AS target, s1.valore + s2.valore AS peso
FROM stat s1, stat s2
WHERE s1.valore > s2.valore

UNION

-- Caso parita: entrambi i versi
SELECT s1.id AS source, s2.id AS target, s1.valore + s2.valore AS peso
FROM stat s1, stat s2
WHERE s1.id <> s2.id
      AND s1.valore = s2.valore;

```

Il template con >= e id diversi e piu breve. UNION e piu chiaro quando vuoi separare bene i casi.

13. Peso degli archi: scegliere la formula giusta

Traccia dice...	Formula consigliata
Numero di elementi in comune	COUNT(DISTINCT elemento)
Numero di volte in cui avviene una relazione	COUNT(*)
Numero di vendite distinte	COUNT(DISTINCT order_id)
Somma delle quantita	SUM(quantity)
Somma degli incassi dei film in comune	SUM(DISTINCT income) oppure CTE film distinti
Somma delle popolarita dei due nodi	stat1.pop + stat2.pop
Differenza tra valori	ABS(x.valore - y.valore)
Distanza temporale	DATEDIFF(data2, data1)
Peso fisso se esiste relazione	1 AS peso oppure grafo non pesato

```

-- Peso come elementi distinti condivisi
COUNT(DISTINCT x.artista) AS peso

-- Peso come somma di valori
SUM(il.Quantity * il.UnitPrice) AS peso

-- Peso come somma di due statistiche gia calcolate
s1.popolarita + s2.popolarita AS peso

-- Peso come differenza assoluta
ABS(s1.valore - s2.valore) AS peso

```

14. CTE: come spezzare query difficili

```

WITH nodi_validi AS (
    SELECT DISTINCT n.id
    FROM names n
    WHERE n.date_of_birth IS NOT NULL
),
film_validi AS (
    SELECT r.movie_id, m.worlwide_gross_income AS income
    FROM ratings r, movie m
    WHERE r.movie_id = m.id
      AND r.avg_rating BETWEEN %s AND %s
      AND m.worlwide_gross_income IS NOT NULL
),
attori_film AS (
    SELECT DISTINCT rm.name_id AS actor_id,
           rm.movie_id,
           fv.income
    FROM role_mapping rm, nodi_validi nv, film_validi fv
    WHERE rm.name_id = nv.id
      AND rm.movie_id = fv.movie_id
)
SELECT af1.actor_id AS id1,
       af2.actor_id AS id2,
       SUM(DISTINCT af1.income) AS peso
FROM attori_film af1, attori_film af2
WHERE af1.movie_id = af2.movie_id
      AND af1.actor_id < af2.actor_id
GROUP BY af1.actor_id, af2.actor_id;

```

Una buona CTE deve avere poche colonne e nomi semplici: id, nodo_id, elemento, peso, valore, anno.

15. Sottoquery, IN ed EXISTS

```
-- IN: prendo solo nodi presenti in una sottoquery
SELECT DISTINCT n.*
FROM names n
WHERE n.id IN (
    SELECT rm.name_id
    FROM role_mapping rm, ratings r
    WHERE rm.movie_id = r.movie_id
    AND r.avg_rating BETWEEN %s AND %s
);

-- EXISTS: tengo un nodo se esiste almeno una relazione
SELECT p.*
FROM products p
WHERE EXISTS (
    SELECT 1
    FROM order_items oi
    WHERE oi.product_id = p.product_id
);

-- NOT EXISTS: nodi senza relazione
SELECT p.*
FROM products p
WHERE NOT EXISTS (
    SELECT 1
    FROM order_items oi
    WHERE oi.product_id = p.product_id
);
```

EXISTS è utile quando la traccia dice "almeno una volta" e non ti serve contare subito.

16. CASE WHEN: creare colonne condizionali

```
-- Etichetta in base al valore
SELECT id,
CASE
    WHEN valore > 10 THEN "alto"
    WHEN valore = 10 THEN "pari"
    ELSE "basso"
END AS classe
FROM stat;

-- Calcolo source/target in base a due valori
SELECT CASE WHEN s1.valore >= s2.valore THEN s1.id ELSE s2.id END AS source,
CASE WHEN s1.valore >= s2.valore THEN s2.id ELSE s1.id END AS target,
s1.valore + s2.valore AS peso
FROM stat s1, stat s2
WHERE s1.id < s2.id
AND s1.valore <> s2.valore;
```

CASE può essere comodo, ma per la parità in grafo orientato spesso è più semplice fare self-join con >=.

17. Query di debug utili quando i pesi non tornano

```
-- Vedere le righe grezze prima del GROUP BY
SELECT rm1.name_id, rm2.name_id, rm1.movie_id
FROM role_mapping rm1, role_mapping rm2
WHERE rm1.movie_id = rm2.movie_id
AND rm1.name_id < rm2.name_id
ORDER BY rm1.name_id, rm2.name_id, rm1.movie_id;

-- Controllare quali elementi sto contando
SELECT id1, id2, GROUP_CONCAT(DISTINCT elemento) AS elementi
FROM ...
GROUP BY id1, id2;

-- Controllare duplicati sospetti
SELECT id1, id2, elemento, COUNT(*) AS quante_righe
FROM ...
GROUP BY id1, id2, elemento
HAVING COUNT(*) > 1;

-- Contare prima le statistiche dei singoli nodi
SELECT nodo_id, COUNT(DISTINCT evento_id) AS valore
FROM ...
```

```
GROUP BY nodo_id
ORDER BY valore DESC;
```

18. Template DAO pronti

18.1 Dropdown

```
@staticmethod
def get_anni():
    conn = DBConnect.get_connection()
    cursor = conn.cursor(dictionary=True)
    query = """
        SELECT DISTINCT r.year
        FROM races r
        ORDER BY r.year DESC
    """
    cursor.execute(query)
    result = [row["year"] for row in cursor]
    cursor.close()
    conn.close()
    return result
```

18.2 Nodi

```
@staticmethod
def get_nodi(parametro):
    conn = DBConnect.get_connection()
    cursor = conn.cursor(dictionary=True)
    query = """
        SELECT DISTINCT n.*
        FROM tabella_nodi n
        WHERE n.campo = %s
    """
    cursor.execute(query, (parametro,))
    result = []
    for row in cursor:
        result.append(Nodo(**row))
    cursor.close()
    conn.close()
    return result
```

18.3 Archi non orientati pesati

```
@staticmethod
def get_archi(parametro):
    conn = DBConnect.get_connection()
    cursor = conn.cursor(dictionary=True)
    query = """
        WITH elementi AS (
            SELECT DISTINCT nodo_id, elemento_condiviso
            FROM ...
            WHERE campo = %s
        )
        SELECT e1.nodo_id AS id1,
               e2.nodo_id AS id2,
               COUNT(DISTINCT e1.elemento_condiviso) AS peso
        FROM elementi e1, elementi e2
        WHERE e1.elemento_condiviso = e2.elemento_condiviso
              AND e1.nodo_id < e2.nodo_id
        GROUP BY e1.nodo_id, e2.nodo_id
    """
    cursor.execute(query, (parametro,))
    result = [(row["id1"], row["id2"], row["peso"]) for row in cursor]
    cursor.close()
    conn.close()
    return result
```

18.4 Archi orientati pesati

```
@staticmethod
def get_archi_orientati(data_inizio, data_fine):
    conn = DBConnect.get_connection()
    cursor = conn.cursor(dictionary=True)
    query = """
        WITH stat AS (
            SELECT oi.product_id AS id,
                   COUNT(DISTINCT oi.order_id) AS valore
        )
```

```

FROM order_items oi, orders o
WHERE oi.order_id = o.order_id
      AND DATE(o.order_date) BETWEEN %s AND %s
GROUP BY oi.product_id
)
SELECT s1.id AS source,
       s2.id AS target,
       s1.valore + s2.valore AS peso
FROM stat s1, stat s2
WHERE s1.id <> s2.id
      AND s1.valore >= s2.valore
"""
cursor.execute(query, (data_inizio, data_fine))
result = [(row["source"], row["target"], row["peso"]) for row in cursor]
cursor.close()
conn.close()
return result

```

19. Cosa fare in SQL e cosa in NetworkX/Python

Operazione	SQL	Python/NetworkX
Nodi del grafo	Si, query SELECT DISTINCT	Aggiunta con add_nodes_from
Archi e peso basati sul DB	Si, meglio già aggregati	Aggiunta con add_edge
Nodi isolati	Query nodi separata	NetworkX li mantiene se aggiunti
Top 5 archi peso maggiore	Possibile con ORDER BY LIMIT	Comodo con sorted(edges(data=True))
Componenti connesse	No	nx.connected_components / weakly_connected_components
Gradi, vicini, predecessori	No dopo il grafo	neighbors, successors, predecessors, degree
Cammini ricorsivi	No	Ricorsione nel Model
Statistiche su campi DB	Si	Solo se già salvate nei nodi/archi

20. Errori tipici e correzione veloce

Errore	Perché succede	Correzione
i.InvoiceId = il.InvoiceId	Collega fattura con riga sbagliata	i.InvoiceId = il.InvoiceId
Date senza apici	SQL le interpreta come sottrazioni	"2016-05-16" o parametro %s
a.id != b.id per grafo non orientato	Duplica A-B e B-A	a.id < b.id
COUNT(*) troppo alto	Join produce duplicati	COUNT(DISTINCT elemento) o CTE DISTINCT
LEAST/GREATEST in grafo orientato	Perdi il verso	source e target espliciti
Filtro su LEFT JOIN nel WHERE	Elimina righe senza match	Sposta filtro nell ON o usa COALESCE
GROUP BY incompleto	Colonne non aggregate non raggruppate	Aggiungi tutte le colonne non aggregate
BETWEEN usato con estremi esclusi	BETWEEN include gli estremi	Usa > e <
NULL confrontato con !=	NULL non si confronta così	IS NOT NULL
Nodi presi dalla query archi	Perdi nodi isolati	Query nodi separata

21. Mini-cookbook: riconosci la frase della traccia

Frase della traccia	Schema da usare
Due nodi sono collegati se hanno X in comune	CTE (nodo, X) + self-join su X + COUNT(DISTINCT X).
Due nodi sono collegati se compaiono nello stesso evento	Self-join con stesso evento_id e id1 < id2.
Arco dal più popolare al meno popolare	CTE popolarità + self-join + confronto >.
In parità inserire entrambi gli archi	Confronto >= con id diversi oppure UNION per casi separati.
Peso pari alla somma delle due popolarità	CTE stat + s1.pop + s2.pop.
Peso pari al numero di vendite distinte	COUNT(DISTINCT order_id).
Estremi inclusi	BETWEEN %s AND %s.
Estremi esclusi	> %s AND < %s.
Almeno una volta	EXISTS oppure COUNT(*) > 0.
Nodi anche se isolati	Query nodi larga + archi separati.
Entro K giorni	DATEDIFF(data2, data1) <= K con data1 < data2.
Top 5	ORDER BY peso DESC LIMIT 5 oppure sorted in Python.